

P3: Sistema de Telemetría

Objetivos	1
Hipótesis / Preguntas	1
Descripción de las métricas	1
Descripción de los eventos	2
Eventos genéricos	2
Eventos específicos (por partida)	3
Cálculo de métricas	5
Telemetría	6
Instrumentalización	6

Corrección:

- los objetivos no pueden ser "medir..." porque estamos definiendo métricas.
- las hipótesis son más acertadas que los objetivos.
- los eventos necesitan algo más de descripción: algunos no tienen timestamp, como vais a representar el tipo de evento (como se va a ver en la traza), tipos y posibles valores.
- ¿Dónde están los eventos genéricos que teneis que implementar? ¿Como se usan los eventos para calcular las métricas? No lo habeis descrito.

Objetivos

Evaluar la capacidad del jugador bajo presión, ver cómo el aumento de la dificultad afecta al rendimiento y a la atención hacia las distracciones.

Se busca comprender cómo la curva de dificultad ascendente (aumento de la velocidad y cantidad de elementos) afecta negativamente al rendimiento de la tarea principal y empeora la gestión de distracciones.

Hipótesis / Preguntas

- Si aumentamos la velocidad y cantidad de las latas generadas, el jugador puede llegar a ignorar tareas secundarias.
- El jugador cometerá más errores si se siente saturado.
- ¿En qué nivel de saturación de latas el jugador ignora las distracciones?

Descripción de las métricas

- **M1.** Número de latas generadas a lo largo de la partida.
- **M2.** Tasa de aciertos / fallos (rendimiento) por jugador y partida.

- **M3.** Distribución de los clicks del jugador (visualizado con un mapa de calor de clicks).
- **M4.** Cantidad de distracciones de cada tipo.
- **M5.** Porcentaje de distracciones resueltas frente al total de distracciones por jugador y por partida.
- **M6.** Tiempo de reacción ante obstáculos por jugador y por partida.

Descripción de los eventos

Eventos genéricos

Parámetros generales:

- Timestamp (Int64) : indica el momento de tiempo en el que se ha lanzado el evento en segundos (tiempo POSIX)
- ID sesión (string): ID único generado mediante una librería de IDs ([shortid](#)).
- ID partida (byte): ID de cada partida, va de 0 a n (siendo n el número de partidas jugadas en una sesión).

Eventos:

- Inicio de sesión. Abrir el juego.
 - Tipo (event_type): "session_start" (string)
 - Atributos:
 - Timestamp
 - ID sesión
- Inicio de partida
 - Tipo (event_type): "match_start" (string)
 - Atributos
 - Timestamp
 - ID sesión
 - ID partida
- Fin de partida
 - Tipo (event_type): "match_end" (string)
 - Atributos
 - Timestamp
 - ID sesión
 - ID partida
- Fin de sesión
 - Tipo (event_type): "session_end" (string)
 - Atributos
 - Timestamp
 - ID sesión

Eventos genéricos					
Evento	Descripción	Tipo	Etiqueta	Atributos	Tipo
Inicio de sesión	Abrir el juego	String	session_start	Timestamp	int64
				ID Sesión	String
Inicio de partida		String	match_start	Timestamp	int64
				ID sesión	String
				ID partida	Byte
Fin de partida		String	match_end	Timestamp	Int64
				ID sesión	string
				ID partida	Byte
Fin de sesión		String	session_end	Timestamp	Int64
				ID sesión	string
				ID partida	Byte

Eventos específicos (por partida)

- Click: recogemos la posición de cada click del jugador para la M3.
 - Tipo: “mouse_click” (string)
 - Atributos
 - ID sesión
 - ID partida
 - Timestamp
 - Posición: coordenadas del ratón al hacer click.
- Aparece lata: Se genera una lata en la cinta
 - Tipo: “can_appears” (string)
 - Atributos
 - ID sesión
 - ID partida
 - Timestamp
 - Color de la lata (Enum { Azul, Rojo, Verde, Rosa })
- Lata encestada: Al lanzar la lata, dónde cae (si ha sido encestada en una caja o fallada (suelo/otro))
 - Tipo: “can_landed” (string)
 - Atributos
 - ID sesión
 - ID partida
 - Timestamp
 - Color lata (Enum { Azul, Rojo, Verde, Rosa })
 - Destino (Enum { Contenedor azul, Contenedor rojo, Basura, Suelo })
- Coger energética: El jugador agarra la bebida energética y la lanza
 - Tipo: “energy_drink_used” (string)
 - Atributos
 - ID sesión

- ID partida
 - Timestamp
 - Destino (Enum { Contenedor azul, Contenedor rojo, Basura, Suelo })
- Aparece distracción: En la escena ocurren diferentes distracciones con las que el jugador tendrá que lidiar, registramos estas distracciones.
 - Tipo: "distraction_spawned" (string)
 - Atributos
 - ID sesión
 - ID partida
 - Timestamp
 - Tipo (Enum { Gato, Mosca, Apagón, Compañeros })
 - ID de la distracción (para saber a qué evento "Desaparece distracción" corresponde) (byte)
- Desaparece distracción: La distracción se va automáticamente o el jugador la quita
 - Tipo: "distraction_despawned" (string)
 - Atributos
 - ID sesión
 - ID partida
 - Timestamp
 - Tipo (Enum { Gato, Mosca, Apagón, Compañeros })
 - ID de la distracción (para saber a qué evento "Aparece distracción" corresponde) (byte)
- Parpadeo: Cuando al jugador le empieza a entrar sueño la pantalla simula el efecto de cerrar y abrir los ojos
 - Tipo "blink" (string)
 - Atributos
 - ID sesión
 - ID partida
 - Timestamp
 - Tipo (bool)
- Perder vida: El jugador encesta una lata en un contenedor de un color distinto al de la lata que ha lanzado
 - Tipo: "life_lost" (string)
 - Atributos
 - ID sesión
 - ID partida
 - Timestamp

Eventos específicos

Evento	Descripción	Tipo	Etiqueta	Atributos	Tipo
Click	Recogemos la posición de cada click del jugador	String	mouse_click	ID sesion	String
				ID partida	byte
				Timestamp	int64
				posicion	vector2D
Aparece lata	Se genera una lata en la cinta	String	can_appears	ID sesion	String
				ID partida	byte
				Timestamp	int64
				Color de la lata	Enum
Lata encestada	Al lanzar la lata, dónde cae (si ha sido encestada en una caja o fallada)	String	can_landed	ID sesion	string
				Id partida	byte
				Timestamp	int64
				Color de la lata	Enum
				Destino	Enum
Coger energética	El jugador agarra la bebida energética y la lanza	string	energy_drink_used	ID Sesion	string
				Id partida	byte
				timestamp	int64
				destino	Enum
Aparece distracción	diferentes distracciones con las que el jugador tendrá que lidiar, registramos	String	distraction_spawned	ID Sesion	string
				Id partida	byte
				timestamp	int64
				tipo	enum
				ID Distraccion	byte
Desaparece distracción	La distracción se va automáticamente o el jugador la quita	String	distraction_destroyed	ID Sesion	string
				Id partida	byte
				timestamp	int64
				tipo	enum
				ID Distraccion	byte
Parpadeo	jugador se empieza a entrar sueño la pantalla simula el efecto de cansancio	String	blink	ID Sesion	string
				Id partida	byte
				timestamp	int64
				Tipo	bool
Perder vida	se genera una lata en un contenedor de un color distinto al de las otras	String	life_lost	ID Sesion	string
				Id partida	byte
				timestamp	int64

Cálculo de métricas

- **M1.** Número de latas generadas a lo largo de la partida.

- Sumando el número de eventos de tipo “can_appears”
- **M2.** Tasa de aciertos / fallos (rendimiento) por jugador y partida.
 - Usando todos los eventos de tipo “can_landed”, sumamos las latas cuyo color y destino corresponden y lo dividimos por la cantidad de eventos de este tipo.
- **M3.** Distribución de los clicks del jugador (visualizado con un mapa de calor de clicks).
 - Gráfica mazo guapa con el evento de “click” y las posiciones del ratón.
- **M4.** Cantidad de distracciones de cada tipo.
 - Usando el evento de “distraction_spawned” y sumando cada tipo.
- **M5.** Porcentaje de distracciones resueltas frente al total de distracciones por jugador y por partida.
- **M6.** Tiempo de reacción ante obstáculos por jugador y por partida.

Instrumentalización

- Clases modificadas
 - GameManager
 - MenuManager
 - LataManager
 - Cursor
 - TiredEvent
 - EventManager
 - EventoJefe
 - EventoJefe2
 - EventoJefe3
 - Gatete
 - LightController
 - Destructor3000
- Clases auxiliares añadidas:
 - Destructor3000Azul
 - Destructor3000Rojo
 - SueloDetector

Enlace al repositorio: <https://github.com/ineprimo/UAJ-P3>.